Project 1 — Cycle Ledger over OPC UA

What you'll build: a small PLC program (a "ledger" that counts completed cycles), then **watch its numbers live in our Crane app** and **push commands to it** (start, stop, reset, target) — all over OPC UA. This is the first end-to-end proof that our software can both **observe** a PLC and **control** it.

> New to the OPC UA words used here (NodeId, namespace, index, security)? Read `../concepts/opcua-explained.md` first — it explains every term in plain English.

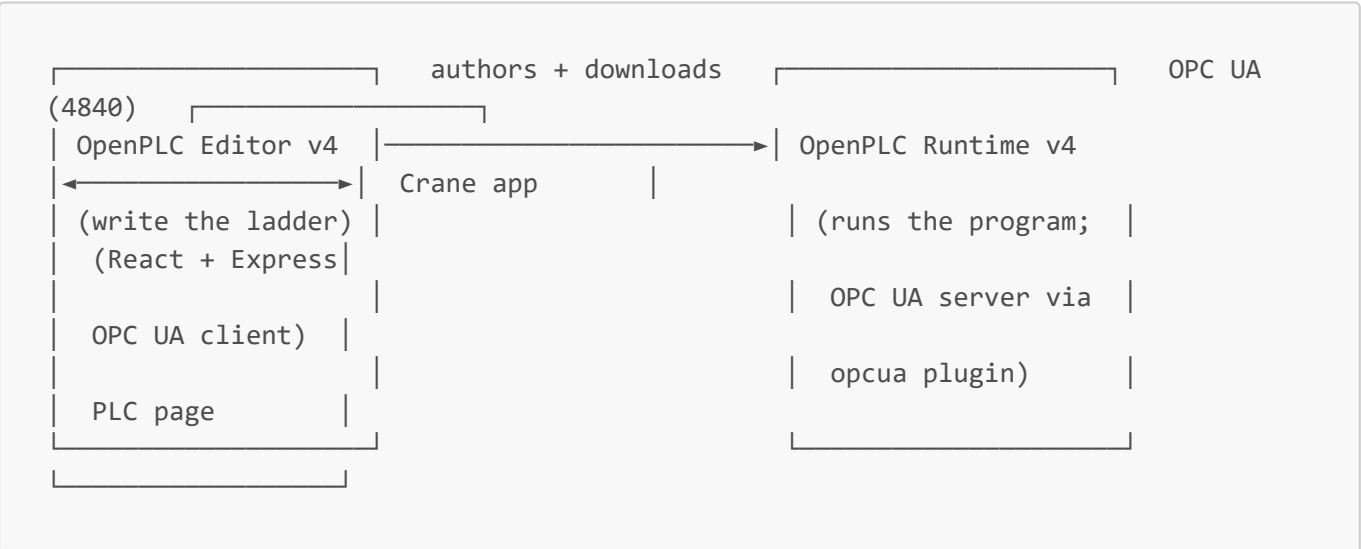
Contents

1. The stack (our version)
2. What the ledger does
3. Variable / tag table (read this before the logic)
4. The PLC program (ladder + Structured Text)
5. Step-by-step build
6. Expose the tags over OPC UA (the mapping)
7. See the data + push commands from our app
8. Do we need to change app code?
9. Glossary (plain language)
10. Progress checklist

---

1. The stack (our version)

Unlike the Modbus/FUXA course stack, ours is **OPC UA end-to-end into our own app**:



- **No Factory I/O, no FUXA needed.** The ledger self-drives with a timer, so you can see it work with just OpenPLC + our app. (Factory I/O is an optional later extension.)
- **Editor v4** = where you *write* the program. **Runtime v4** = the engine that *runs* it and serves OPC UA. Our **Crane app** = the operator screen (replaces FUXA).
- Everything is on `localhost` for now; later the same app points at a Siemens S7-1500 by changing only the endpoint + security (see `../concepts/plc-opcua.md`).

2. What the ledger does

A **ledger** here = a running count of completed cycles, that an operator can control.

- While **run** is on, the program completes one "cycle" every second (a timer pulse) and **increments a counter** (`cycle_count`).
- When the count reaches a **target** the operator set, a **done** flag turns on.
- The operator can **reset** the count to zero, and **stop** at any time.
- Our app **reads** `cycle_count`, `running`, `done` (PLC → software) and **writes** `run`, `reset`, `target` (software → PLC).

Think of it as counting crane trips / tank transfers: a real, resettable operations log with a goal — a natural first "ledger."

---

3. Variable / tag table

Declare these in the program. **Locations** are what make a tag a real memory point.

Tag	Type	Location	Direction	Meaning
`run`	BOOL	`%QX0.0`	software → PLC	master start/stop (true = running)
`reset`	BOOL	`%QX0.1`	software → PLC	true = zero the counter
`target`	INT	`%QW1`	software → PLC	goal count for `done`
`cycle_count`	INT	`%QW0`	PLC → software	**the ledger** — cycles completed
`running`	BOOL	`%QX0.2`	PLC → software	status echo of `run`
`done`	BOOL	`%QX0.3`	PLC → software	count reached target
`blink`	TON	—	internal	1-second oscillator
`pulse`	R_TRIG	—	internal	one-shot each second
`ctr`	CTU	—	internal	the up-counter

> Software-written tags (`run`, `reset`, `target`) sit in the output/register area but the program **never assigns them**, so values you write over OPC UA persist. Tags the program drives (`cycle_count`, `running`, `done`) are read-only from the app's point of view.

---

4. The PLC program

You can build this as **Ladder (LD)** — recommended for learning — or paste it as **Structured Text (ST)** which is exact and fast. Both produce the same result.

Structured Text (copy-paste into an ST POU)

```
(* Cycle Ledger — self-driving counter with software control *)
blink(IN := run AND NOT blink.Q, PT := T#1S);   (* 1 Hz oscillator, only while run
*)
pulse(CLK := blink.Q);                           (* one TRUE scan per second *)
ctr(CU := pulse.Q, R := reset, PV := target);    (* count up; reset zeroes it *)
cycle_count := ctr.CV;                           (* expose the count *)
running := run;                                   (* status echo *)
done := ctr.Q;                                    (* reached target *)
```

Variable block for the ST POU:

```
VAR
  run         AT %QX0.0 : BOOL;
  reset       AT %QX0.1 : BOOL;
  running     AT %QX0.2 : BOOL;
  done        AT %QX0.3 : BOOL;
  cycle_count AT %QW0   : INT;
  target      AT %QW1   : INT := 10;
  blink : TON;
  pulse : R_TRIG;
  ctr   : CTU;
END_VAR
```

Ladder equivalent (4 rungs)

- **Rung 1 — 1 Hz tick (only while running):** `TON` block `blink`, `IN` = `run` AND (NC contact of `blink.Q`), `PT` = `T#1s`.
- **Rung 2 — one-shot:** `R_TRIG` block `pulse`, `CLK` = `blink.Q`.
- **Rung 3 — count:** `CTU` block `ctr`, `CU` = `pulse.Q`, `R` = `reset`, `PV` = `target`; wire `ctr.CV` → `cycle_count`, `ctr.Q` → `done` coil.
- **Rung 4 — status:** contact `run` → coil `running`.

---

5. Step-by-step build

> Coaching loop: do one step, screenshot it, and I'll confirm before the next — same style as your 30-day course. Put screenshots in `docs/projects/assets/`.

1. **Open OpenPLC Editor v4** → **New project** → name it `CycleLedger` → pick **Structured Text** (or Ladder) → set the **cyclic task** to `T#20ms` (default).
2. **Declare the variables** from §3 in the POU variable table (set the **Location** column exactly: `%QX0.0`, `%QX0.1`, `%QX0.2`, `%QX0.3`, `%QW0`, `%QW1`).
3. **Enter the logic** from §4 (paste the ST, or build the 4 rungs).
4. **Compile** (Editor checks it). Fix any red errors before moving on.
5. **Connect to the Runtime** (Editor → connect; login `openplc` / `openplc` if asked) and **Download / Transfer** the program. Confirm the Runtime shows **PLC: RUNNING**.

    - Behind the scenes the Runtime regenerates `core/generated/` and recompiles `build/libplc_*.so`. (See `../openplc_opcua/enable-opcua-plugin.md`.)
6. **Confirm the opcua plugin is on** (`plugins.conf` line `opcua,…,1,…`). If you just enabled it, restart the Runtime **while stopped** (plugin config is read at startup).

---

6. Expose the tags over OPC UA (the mapping)

This is the step that trips everyone up — the `index` numbering. **Don't guess it.**

1. After the download, the exact indices are written to `…\OpenPLC Runtime\openplc-runtime\core\generated\VARIABLES.csv`.
2. **The OPC UA `index` = the variable's `debug.c` order (0-based), which is `VARIABLES.csv` line number minus 1** (that file lists the program instance at line 0, shifting everything by one). Full explanation: `../concepts/opcua-explained.md` §6.
3. Edit `core\src\drivers\plugins\python\opcua\opcua.json` → put one entry per tag you want exposed, each with its `datatype`, correct `index`, and `"permissions": {"viewer": "rw", …}` (anonymous sessions get the `viewer` role; `rw` is needed to let the app write).
4. **Restart the Runtime.** NodeIds are then assigned `ns=2;i=1`, `ns=2;i=2`, … **in the order the tags appear in `opcua.json`**.
5. **Verify by browsing** the live server (UaExpert, or `npx opcua-commander -e opc.tcp://localhost:4840/openplc/opcua`) — confirm each NodeId's value/type before trusting it.

> **Shortcut:** after you download the program, tell me and I'll read your `VARIABLES.csv`, generate the exact `opcua.json`, restart the runtime, and browse to confirm the NodeIds — same as we did for the current `test_read`/`test_coil` demo.

---

7. See the data + push commands from our app

In the Crane app → **PLC** page (log in as super-admin):

1. **Connect:** Endpoint `opc.tcp://localhost:4840/openplc/opcua`, Security **None**, Policy **None** → **Connect**.
2. **See the data (PLC → software):** set **Read NodeId** to `cycle_count`'s NodeId (e.g. `ns=2;i=1`). The **Live read** tile shows the count.
3. **Push commands (software → PLC):** in **Send command**
    - Write `target` (Int16, e.g. `20`) → sets the goal.
    - Write `run` (Boolean `true`) → the counter starts climbing (watch the Live read move).
    - Write `run` (Boolean `false`) → it stops.
    - Write `reset` (Boolean `true`, then `false`) → the count returns to 0.
4. Switch the Read NodeId to `running` or `done` to watch those flags flip.

That's the whole loop: **observe** a live PLC value and **command** the PLC, both over OPC UA.

---

8. Do we need to change app code?

No — not for this project. The PLC page already reads any NodeId and writes any NodeId + data type, and the security dropdown makes it generic across PLCs. This project is entirely **PLC program + `opcua.json`**

mapping.

Optional enhancement (nice-to-have): the Live read currently watches **one** NodeId at a time. A "ledger" naturally wants several values at once (count + running + done + target). If you want, I can add a small **multi-tag watch table** to the PLC page so you see all of them together — that *would* be an app code change (a new component + a batch-read, still generic). Say the word and I'll build it as Project 1's follow-up.

---

9. Glossary

Only terms this project introduces (OPC UA terms are in `../concepts/opcua-explained.md` §9):

- **Ladder logic (LD)** — the wiring-diagram style PLC language: power flows left→right through contacts (conditions) to coils (outputs). Everyday example: a stairwell light that turns on if *either* switch is flipped — the two switches are contacts, the bulb is the coil.
- **Rung** — one horizontal line of ladder logic = one rule. Like one "if this, then that."
- **Located variable** — a tag pinned to a specific PLC memory address (`%QX0.0`, `%QW0`). Like giving a contact in your phone a fixed speed-dial slot instead of a random one.
- **Coil (`%QX`)** — a single on/off output bit. Everyday example: a light switch position.
- **Holding register (`%QW`)** — a 16-bit number slot. Everyday example: the number on a mechanical tally counter.
- **Cyclic task (`T#20ms`)** — the PLC re-runs the whole program every 20 ms, forever. Like a chef re-checking every pot every few seconds.
- **TON (on-delay timer)** — output turns on only after the input stayed on for a set time. Everyday: a bathroom fan that keeps running a bit, or a microwave that waits before starting.
- **R_TRIG (rising-edge one-shot)** — fires a single pulse the instant a signal goes from off to on. Everyday: a doorbell that dings once when pressed, not continuously.
- **CTU (up counter)** — adds 1 each time its input pulses; has a reset and a preset (target). Everyday: a golf stroke counter you click and can zero.
- **Setpoint / target** — the number you want the process to aim for. Everyday: the temperature you set on a thermostat.

---

10. Progress checklist

Mark each ☐ → 🔄 → ✅ as you go (I'll confirm before advancing):

- ☐ Editor v4 project created with the cyclic task set
- ☐ All variables declared with correct **Locations**
- ☐ Logic entered (ST pasted or 4 rungs built) and **compiles clean**
- ☐ Downloaded to Runtime; **PLC: RUNNING**; opcua plugin enabled
- ☐ `opcua.json` mapped from `VARIABLES.csv` (correct indices) + `viewer: rw`; Runtime restarted
- ☐ NodeIds verified by browsing the live server
- ☐ App connects (None security) and **Live read shows `cycle_count`**
- ☐ Wrote `target`, `run`, `reset` from the app and saw the counter respond
- ☐ (optional) Multi-tag watch table added to the PLC page

When each step is done, we also append a one-line entry to `../software-flow.md`'s build log so the project history stays current.